Limitaciones del proyecto y casos de uso del sistema

> Borrador de trabajo para el primer entregable posterior al estado del arte. Estado al 2026-10-12. Es **material para decidir**, no decisiones tomadas: cada limitación tiene una recomendación y un casillero «Decisión» para completar entre Paola y Jero. Los casos de uso están escritos suponiendo que se aceptan las recomendaciones; si alguna cambia, la sección 4 dice qué casos se ven afectados.

← Volver a la puerta de entrada

Fuentes: informe de estado del arte (en adelante «el informe»), letra del proyecto, plan de pruebas, comparativa y las fichas de las pruebas. Cuando un argumento sale de una prueba propia se cita su ID (A1, D2, …); cuando sale del informe, la sección.

---

1. Qué falta para el entregable

No tengo la plantilla formal del entregable, así que la lista sale de lo que la letra pide como «próximos pasos» (sección 6) y de lo que el informe deja abierto (cap. 3: «el diseño detallado, especificación de requerimientos y despliegue final quedan proyectados para las etapas posteriores»). **Confirmen con el tutor qué secciones lleva.**

#	Pieza	Estado	Qué falta
1	Alcance y limitaciones	No existe	Este documento, sección 2
2	Caso de estudio (base de ejemplo)	Pagila elegida para las pruebas, no para el proyecto	Decidir si alcanza (L5)
3	Requisitos / casos de uso	No existe	Sección 3 de este documento
4	Arquitectura refinada	Seis capas en el informe (cap. 3)	Bajar a componentes y artefactos concretos usando los hallazgos de las pruebas
5	Pila tecnológica confirmada	Candidata en el informe (Tabla 3.1)	Confirmar o cambiar cada fila con evidencia P (ver sección 5)
6	Métricas del caso de estudio y umbrales del gate	Métricas listadas (Tablas 2.11 y 2.20)	Fijar **umbrales** concretos; sin eso el gate no es ejecutable
7	Aporte original	Cinco candidatas (5.2)	Elegir una o dos (L18)
8	Plan de trabajo / cronograma	No existe	Depende de 1–7

#	Pieza	Estado	Qué falta
9	Cierre de las pruebas de herramientas	A, D y E con resultados; B, C, F, G sin resultados	Ver sección 5: qué pruebas siguen haciendo falta y cuáles pierden sentido según las limitaciones

2. Limitaciones a definir

Ordenadas por cuánto condicionan al resto. Las cuatro primeras (motor, hardware, caso de estudio, datos reales) hay que cerrarlas antes que las demás.

Resumen

ID	Dimensión	Recomendación corta
L1	Motor de base de datos	Solo PostgreSQL (≥ 16), origen y destino
L2	Origen ≠ destino	Mismo motor; sin migración entre motores
L3	Hardware	Portátiles Apple Silicon sin GPU NVIDIA; todo por CPU
L4	Escala de la base	Acotar a lo medido: decenas de tablas, orden 10^5–10^6 filas
L5	Caso de estudio	Pagila + un segundo esquema no visto por los LLM
L6	Datos productivos reales	No hay acceso; se dice explícito
L7	Características del esquema soportadas	Lista cerrada (ver tabla)
L8	Tipos de datos soportados	Escalares sí; texto libre solo detectar y redactar; resto fuera o regla fija
L9	Idioma y formatos locales	Español e inglés; identificadores uruguayos
L10	Modelo de lenguaje	Solo local; qué ve el modelo, acotado
L11	Modo de operación	Lote, por línea de comandos, con archivos de configuración; sin interfaz gráfica
L12	Utilidad para testing	Suite de referencia propia sobre el caso de estudio
L13	Alcance de la garantía de privacidad	Se mide el riesgo; no se certifica anonimato
L14	Reproducibilidad	Reproducible por artefactos, no siempre por semilla
L15	Gestión de secretos	Mínima y documentada
L16	Herramientas comerciales	Solo como referencia
L17	Volumen de salida	Igual o menor que el origen; sin «upsampling»
L18	Aporte original	Elegir una o dos de las cinco candidatas

L1. Motor de base de datos

Por qué hace falta acotarlo. La capa 0 (introspección) y la capa 3 (transformación determinista) dependen del motor. `information_schema` es estándar, pero las cosas que importan no lo son: particiones, dominios, `pg_constraint` para FK que no aparecen bien en `information_schema`, secuencias, columnas generadas. E3 mostró que en Pagila 49 de 55 particiones de `payment` no tienen FK: el detalle físico del motor afecta directamente la integridad.

Opción	A favor	En contra
A. Solo PostgreSQL	Todas las pruebas ya están en PostgreSQL (Pagila, Neon, Anonymizer). PostgreSQL Anonymizer pasa a ser la pieza principal de la capa 3 determinista (Tabla 3.1 del informe lo condiciona a «si el destino es PostgreSQL»). Es el motor mejor cubierto por el ecosistema abierto (Tabla 2.17).	No se demuestra portabilidad. Pierde relevancia para organizaciones con SQL Server u Oracle.
B. PostgreSQL + MySQL/MariaDB	Muestra que la arquitectura no está atada a un motor.	Duplica la capa 0 y obliga a usar Faker + tokenización propia en vez de Anonymizer en MySQL (el enmascarado de MySQL es solo Enterprise, Tabla 2.7). Duplica casos de prueba.
C. Agnóstico vía SQLAlchemy	Atractivo en papel.	Es la opción que más trabajo esconde: SQLAlchemy no abstrae particiones ni seguridad por etiquetas; terminás con código por motor igual.

Recomendación: A, con dos matices:

1. Diseñar la capa 0 detrás de una interfaz (en C#: un `IIntrospector` con una implementación `PostgresIntrospector`) para que la extensión a otro motor sea trabajo futuro documentado y no reescritura.
2. Que la transformación determinista **no dependa exclusivamente** de PostgreSQL Anonymizer: dejar Faker + tokenización propia como implementación alternativa del mismo contrato. Así, si B1 muestra problemas con Anonymizer, el diseño no se cae.

Versión: PostgreSQL ≥ 16. Ojo: la imagen de Anonymizer está en PostgreSQL 18 y Neon también (README, ficha de Neon); el contenedor común está en 16. Conviene fijar una versión única para el caso de estudio.

Qué se pierde y se dice como limitación: SQL Server, Oracle, Db2, SAP ASE y servicios gestionados que no permiten extensiones (Aurora, Tabla 2.17). Con PostgreSQL gestionado sin la extensión `anon` (por ejemplo, algunos planes de nube) se usaría la alternativa Faker + tokenización.

> **Decisión:**

L2. Origen y destino en el mismo motor

Consecuencia de L1, pero conviene escribirla aparte porque es una pregunta que el tribunal hace. **Recomendación:** origen y destino PostgreSQL; el producto es un *dump* SQL (o una base) cargable en la misma versión. Fuera de alcance: migrar entre motores o cambiar el esquema en el camino.

> **Decisión:**

L3. Hardware

Dato, no decisión: los equipos son una Mac M5 Pro de 16 GB (Jero) y una M1 de 8 GB (Paola), sin GPU NVIDIA (plan). Consecuencias, con su fuente:

- MOSTLY AI no usa la GPU de Apple (MPS): entrena por CPU (D2, P).
- Quedan fuera los generadores por difusión (TabDDPM, ClavaDDPM; Tabla 2.13 «GPU necesaria») y NeMo Safe Synthesizer (requiere A100 80 GB+).
- El modelo de lenguaje local queda acotado a lo que entra en 16 GB compartidos con Docker. D2 llegó a 8,1 GB de pico **con Docker ocupando otros 8 GB**: no se puede correr la síntesis y un LLM grande a la vez. Orden práctico: modelos de hasta ~14B parámetros cuantizados (A2a usó Gemma 12B y 26B fuera del repo; habría que ver si el 26B entra junto con lo demás).
- La M1 de 8 GB probablemente no pueda correr la síntesis de Pagila completa. **Hay que medirlo**: si no puede, es una limitación de reparto de trabajo, no del proyecto.

Pregunta abierta: ¿la facultad da acceso a algún servidor (ClusterUY, por ejemplo)? Si sí, cambia L4; si no, L3 se escribe como limitación tal cual y se descartan explícitamente las técnicas que exigen GPU.

> **Decisión:** ¿hay acceso a otro equipo?

L4. Escala de la base

Por qué. Sin un tope, cualquier crítica de escalabilidad es válida. Con un tope medido, es una limitación honesta.

Lo que hay medido (D2): Pagila completa (~106 000 filas generadas, 15 tablas + 55 particiones) llevó 8,5 min de entrenamiento, 12,1 min de generación y 8,1 GB de pico en la M5 Pro.

Recomendación: declarar el alcance como «bases de decenas de tablas y del orden de 10^5 a 10^6 filas», y **medir la curva** (tiempo y memoria vs. filas) con dos o tres tamaños antes de fijar el número. El subsetting (capa 2) es justamente lo que permite trabajar con bases más grandes: el límite aplica al subconjunto, no al origen. Eso conviene decirlo así.

> **Decisión:** tope concreto después de medir

L5. Caso de estudio

Es la decisión abierta más importante después del motor.

Problemas de usar solo Pagila:

1. **No tiene datos personales reales**: son nombres inventados, no hay cédulas, no hay texto libre con PII.

2. **Es pública y los LLM la conocen.** D5 lo mostró: Fabricate generó el Sakila clásico (2005, 2 tiendas) en vez del esquema v4 que se le pasó, y consultó la base pública en internet. Para medir la clasificación con un LLM (A2, A6, G1) eso es **contaminación**: el modelo puede acertar porque recuerda Pagila, no porque razone sobre el esquema.
3. **No tiene una aplicación con tests**, que es lo que pide la utilidad para testing (L12).
4. Es chica y de un solo dominio.

Lo que sí tiene a favor: ciclos (`store` ↔ `staff`), particiones, una FK no declarada, tablas de catálogo, `bytea`, texto libre (`film.description`) y todas las pruebas ya hechas sobre ella.

Opción	Comentario
A. Solo Pagila	Lo más barato. Las limitaciones 1–3 quedan como limitaciones del trabajo. Debilita fuerte el aporte si incluye la clasificación con LLM.
B. Pagila + un segundo esquema	Pagila para comparar con las pruebas; un segundo esquema con dominio sensible (salud, banca, RR. HH.) y columnas en español con identificadores uruguayos para validar sin contaminación.
C. Pagila con columnas renombradas	Control barato contra la contaminación: el mismo esquema con nombres neutros (`t1.c3`). A1 ya hizo algo así con Presidio. No resuelve 1, 3 ni 4.

Recomendación: B + C. Para el segundo esquema hay dos caminos: uno público menos conocido, o **uno propio diseñado por ustedes** y poblado con datos ficticios uruguayos (Faker no tiene `es_UY`, informe 2.2.3: hay que armar generadores de cédula con dígito verificador, celulares, departamentos). El propio tiene la ventaja de que ningún LLM lo vio y de que pueden meterle a propósito los casos difíciles (FK virtuales, PK compuestas, texto libre con PII, columnas con nombre ambiguo como el `Ticket` del informe). La desventaja es que «lo diseñamos para que funcione» es una crítica fácil: habría que diseñarlo **antes** de implementar la clasificación y congelarlo.

No sugiero un esquema público concreto sin verificar su licencia y su estado actual; si van por ahí, lo buscamos.

> **Decisión:**

L6. Sin datos productivos reales

No van a tener una base productiva. Escribirlo como limitación explícita, con su consecuencia: **las métricas de privacidad se calculan sobre datos de ejemplo** y no permiten afirmar que el proceso sea suficiente sobre datos reales. Esto también protege al proyecto del lado legal (no hay tratamiento de datos personales reales).

> **Decisión:** (es más confirmación que decisión)

L7. Características del esquema soportadas

Característica	Recomendación	Fuente
FK declaradas, simples y compuestas	Dentro	—

Característica	Recomendación	Fuente
PK compuestas y restricciones UNIQUE	Dentro (obligatorio verificarlas)	D2: 269 duplicados en tablas de relación
Ciclos de FK	Dentro	Pagila `store` ↔ `staff`, plan C1
Tablas particionadas	Dentro	D5 perdió `payment` entera; E3: FK faltantes en particiones
FK virtuales (no declaradas)	Dentro: el sistema **propone** por nombre y contenido; una persona confirma	E3, informe 2.4.3
Tablas de catálogo / referencia	Dentro: se copian, no se sintetizan	D2: categorías e idiomas salieron todos `_RARE_`
CHECK, NOT NULL, dominios	Dentro	—
Columnas generadas / calculadas	Dentro: se excluyen y se recalculan al cargar	Tabla 2.19
Triggers, vistas, funciones	Se copian como esquema; no se analiza su lógica	—
Referencias polimórficas	**Fuera**	Informe 2.4.3: no lo resuelve ningún generador abierto
Reglas que viven en el código de la aplicación	**Fuera**, salvo que se declaren a mano como restricciones de negocio	Informe 2.4.3
Varios esquemas / varias bases relacionadas	Fuera (un esquema por corrida)	—

> **Decisión:**

L8. Tipos de datos

Tipo	Recomendación
Numéricos, fechas, texto corto, ENUM	Dentro
Texto libre	**Detectar y redactar**, no sintetizar. A7 mostró que hace falta detección en texto además de por columna. Sintetizar texto con un LLM es otro proyecto.
BYTEA / BLOB	Reemplazo por un valor de relleno del mismo tamaño (Tabla 2.19). `staff.picture` es el caso.
JSON / JSONB, ARRAY	Fuera, o solo reglas deterministas sobre claves declaradas a mano.

Tipo	Recomendación
Geoespaciales	Fuera.
Columnas cifradas	Se documentan y se excluyen.
Hashes de contraseña	Dentro como caso especial: reemplazo por un hash fijo conocido. Ojo: ningún detector probado marcó `staff.password` (A1) y MOSTLY copió el hash constante (D2).

> **Decisión:**

L9. Idioma y formatos locales

Recomendación: datos en español e inglés; identificadores uruguayos (cédula con dígito verificador, celulares, RUT) con reconocedor propio. Sale de A1 (el modelo en inglés de Presidio marca toda la frase como `PERSON`) y A7 (la cédula sale como `NUMERIC_PII` genérico). Otros países: fuera. Es además lo que da sentido local al proyecto (Ley 18.331).

> **Decisión:**

L10. Modelo de lenguaje

Dos decisiones en una:

1. **Dónde corre.** Recomendación: solo local, por la restricción on-premise. Una API comercial solo como **punto de comparación** en el experimento (si eligen la candidata «SLM local vs. LLM comercial» como aporte), y **solo con metadatos**, nunca valores (Tabla 4.1 del informe).
2. **Qué ve el modelo.** Opciones de menor a mayor exposición:
   - nombres de tabla y columna, tipos, claves (A2a);
   - lo anterior + estadísticos agregados (cardinalidad, % nulos, largo promedio, patrón de formato);
   - lo anterior + una muestra de valores (A2b).

   Recomendación: definirlo **después de A2b**, con números. Si los estadísticos rinden casi como la muestra, la muestra queda fuera y el argumento de privacidad es mucho más fuerte.

Además, por lo que dice el informe (3.3) y el principio «el LLM propone, el sistema valida»: el modelo **nunca** ejecuta SQL, solo devuelve JSON validado contra un esquema. Eso no es limitación sino diseño, pero conviene que figure.

> **Decisión:** (1) (2) después de A2b

L11. Modo de operación

Recomendación: proceso por lotes, ejecutado por línea de comandos, con configuración en archivos (YAML) y la revisión humana hecha **editando un archivo** (el catálogo) y no en una interfaz gráfica. Fuera de alcance:

- interfaz web;

- integración real con un pipeline de CI/CD (se puede mostrar que el comando es invocable desde uno);
- refrescos periódicos automáticos y contabilidad del presupuesto ε entre refrescos (riesgo de la Tabla 4.1; se documenta, no se implementa);
- multiusuario y control de acceso.

Una interfaz gráfica consume mucho tiempo y no aporta a ninguna de las preguntas de investigación.

> **Decisión:**

L12. Utilidad para testing

Es la que más trabajo esconde. La Tabla 2.20 del informe pide suite de regresión, cobertura de código y planes de ejecución. **Pagila no tiene aplicación ni tests**, y el segundo esquema tampoco los va a tener salvo que los hagan.

Opción	Comentario
A. Suite de referencia propia	Un conjunto fijo de consultas de negocio con aserciones («todo pago tiene alquiler», «ningún cliente activo sin dirección», reportes con resultado no vacío) + `EXPLAIN` de esas consultas para comparar planes. Barato y medible.
B. Mini-aplicación con tests	Permite medir cobertura de código de verdad. Más caro; es casi otro proyecto.
C. Aplicación de terceros sobre el esquema	Si existiera una app abierta con tests sobre Pagila/Sakila. Habría que buscarla y verificarla.

Recomendación: A, y dejar «cobertura de código» como limitación explícita (no hay aplicación). Esto además encaja con la candidata de aporte «métricas de utilidad orientadas a testing» (L18). Es lo que mide F1, que hoy está sin responsable.

> **Decisión:**

L13. Alcance de la garantía de privacidad

Recomendación: el sistema **mide y reporta el riesgo** (Anonymeter, DCR contra control, coincidencias exactas, k-anonimato con ARX sobre cuasi-identificadores) y **bloquea** por umbral, pero **no certifica** que el resultado sea anónimo en sentido legal. Coherente con el principio «sintético no implica anónimo» y con el EDPB 02/2026. La salida se trata como dato personal salvo evaluación del responsable.

Dos cosas que las pruebas ya muestran y conviene escribir como limitación de la medición:

- Anonymeter no es determinista: se informa con intervalo, no al tercer decimal (E3, E5).
- Una base generada **sin ver datos** dio 0,18 de singling out (E5): ese es el ruido de fondo; por debajo, la métrica no distingue herramientas.

Además hay que **fijar los umbrales del gate** (pieza 6 de la sección 1). Sin umbral, «gate bloqueante» es una intención.

> **Decisión:** umbrales

L14. Reproducibilidad

Hay una tensión con lo que dice el informe que tienen que resolver. El informe (anexo A y 3.3) dice que misma semilla + mismo catálogo + mismo snapshot da el mismo resultado. Pero D2 registró que **MOSTLY AI no tiene semilla** y las corridas no se pueden reproducir exactamente, y Anonymeter tampoco es determinista.

Opciones:

- **A.** Redefinir la reproducibilidad como **reproducibilidad por artefactos**: se guarda el generador entrenado (MOSTLY lo exporta, informe 2.2.7) y la base generada con su hash; cualquiera puede repetir la validación sobre el mismo artefacto. Las etapas deterministas (introspección, subsetting, enmascarado) sí se reproducen bit a bit.
- **B.** Volver a verificar si el SDK expone alguna semilla (puede haber cambiado de versión, o puede haberse pasado por alto). Si existe, se mantiene lo del informe.

Recomendación: hacer B (es barato) y escribir A como la definición de todos modos, porque es más honesta y cubre también a Anonymeter. Esto modifica lo que dice el informe: díganlo explícitamente en el entregable.

> **Decisión:**

L15. Gestión de secretos

La sal de la tokenización y la tabla de mapeo de seudonimización son el punto más sensible (Tabla 4.1: «revierte todo el proceso de una vez»). **Recomendación:** secretos por variable de entorno o archivo fuera del repo, referenciados en el manifiesto por identificador (como ya propone el anexo A); integración con un gestor de secretos o KMS, fuera de alcance. Se dice como limitación.

> **Decisión:**

L16. Herramientas comerciales

Ya está decidido en el informe: Tonic y Delphix solo como referencia. Lo que queda por decidir es **cuánto más se invierte en ellas**. Las pruebas que requieren el trial de Tonic Structural (A8, B4, C2, E6, G3) cuestan tiempo y el trial dura 14 días. **Recomendación:** hacer como máximo C2 (subsetting, porque es la pieza que hay que construir y Tonic es la línea base nombrada en la Tabla 3.1) y A8 si sobra tiempo dentro del mismo trial; el resto, desestimarlo.

> **Decisión:**

L17. Volumen de salida

Recomendación: la base de preproducción tiene volumen **igual o menor** que el origen (subconjunto + síntesis a escala del subconjunto o del original). Generar **más** volumen que el original (para pruebas de carga) queda fuera: cambia las métricas de privacidad y de planes de ejecución.

> **Decisión:**

L18. Aporte original

No es una limitación técnica pero acota todo lo demás. Las cinco candidatas del informe, con lo que cada una exige:

Candidata	Exige	Encaje con las recomendaciones
Métricas de utilidad orientadas a testing	L12 opción A bien hecha	Alto: se apoya en la suite de referencia
Capa de restricciones de negocio sobre un generador abierto	Formalizar restricciones + remuestreo; D2 y D5 dan los casos que fallan	Alto: los hallazgos ya muestran el hueco (duplicados, fechas fuera de partición)
Identificación por topología del grafo	Experimento de ataque sobre la estructura	Medio: Pagila es chica para esto
Trade-off ε vs. utilidad de testing	DP en MOSTLY + suite de referencia	Medio: depende de L12
SLM local vs. LLM comercial para clasificación	L5 con esquema no contaminado	Medio: sin el segundo esquema, el resultado es débil

Recomendación: restricciones de negocio + métricas de utilidad para testing. Se refuerzan entre sí (las restricciones se validan con la suite) y ya tienen evidencia de las pruebas.

> **Decisión:**

---

3. Casos de uso

Suponen las recomendaciones de la sección 2. Formato: cada caso dice quién lo dispara, qué entra, qué sale (**siempre un artefacto versionado**), qué hace la IA y cómo se verifica que funcionó. La IA aparece solo donde propone; nunca ejecuta.

Actores

Actor	Rol
Responsable de datos	Configura el origen, revisa y aprueba el catálogo, aprueba el resultado. En un equipo real: DBA o dueño de los datos.
Revisor de privacidad	Fija umbrales del gate, revisa el reporte de riesgo. En un equipo real: DPO. Para el proyecto puede ser la misma persona.
Equipo de testing	Consume la base de preproducción y aporta la suite de referencia.
Orquestador	El sistema. Ejecuta las etapas, valida todo lo que propone el modelo.
Modelo de lenguaje local	Propone clasificaciones, FK virtuales y restricciones. Solo recibe metadatos (L10).

Lista y prioridad

Prioridad: **M** = imprescindible para el prototipo; **D** = deseable; **F** = futuro (se describe, no se implementa).

CU	Nombre	Capa	Actor principal	Prioridad
CU-01	Registrar el origen e introspeccionar el esquema	0	Responsable de datos	M
CU-02	Proponer relaciones no declaradas	0	Orquestador + LLM	M
CU-03	Clasificar columnas	1	Orquestador + LLM	M
CU-04	Revisar y congelar el catálogo	1	Responsable de datos	M
CU-05	Declarar restricciones de negocio	3	Responsable de datos (LLM propone)	M
CU-06	Extraer un subconjunto	2	Responsable de datos	M
CU-07	Separar el conjunto de control	2/4	Orquestador	M
CU-08	Transformar (enrutamiento por clase de columna)	3	Orquestador	M
CU-09	Validar y aplicar el gate	4	Orquestador / Revisor	M
CU-10	Emitir el manifiesto	5	Orquestador	M
CU-11	Entregar la base a preproducción	—	Responsable de datos	M
CU-12	Reproducir o auditar una corrida	5	Revisor	D
CU-13	Detectar deriva de esquema y refrescar	0/1	Responsable de datos	D
CU-14	Redactar texto libre	3	Orquestador	D
CU-15	Pedido en lenguaje natural → plan	—	Equipo de testing	F

Detalle

CU-01 · Registrar el origen e introspeccionar

- **Entrada:** cadena de conexión (por variable de entorno, nunca en archivos), esquema a procesar.
- **Hace:** lee `information_schema` y `pg_catalog`: tablas, columnas, tipos, PK, FK, UNIQUE, CHECK, particiones, columnas generadas, cardinalidades y conteos.

- **Sale:** `grafo.json` (nodos = tablas, aristas = FK, marcando ciclos y tablas de catálogo candidatas) y hash del esquema.
- **IA:** ninguna. Determinista.
- **Se verifica:** sobre Pagila, encuentra los ciclos y las 55 particiones; dos corridas dan el mismo hash.

CU-02 · Proponer relaciones no declaradas

- **Entrada:** `grafo.json` + estadísticos por columna.
- **Hace:** candidatas por nombre (`*_id`, `manager_staff_id`) y por contenido (inclusión de valores). El LLM puede sugerir candidatas por semántica; **el orquestador verifica la inclusión de valores** antes de proponerla.
- **Sale:** lista de FK virtuales propuestas, con evidencia; pasa a CU-04 para confirmación.
- **Se verifica:** encuentra `store.manager_staff_id` y la FK de las particiones de `payment` (E3).

CU-03 · Clasificar columnas

- **Entrada:** grafo + estadísticos + (según L10) muestra local.
- **Hace:** cascada del informe: reglas (Presidio + reconocedores uruguayos) → NER local → LLM local con contexto de esquema. Excluye de entrada las claves (A1). Cada columna sale con clase, estrategia propuesta, confianza y qué detector la marcó.
- **Clases mínimas:** identificador directo, cuasi-identificador, sensible, clave (PK/FK), catálogo, texto libre, no sensible.
- **Sale:** `catalogo.propuesto.yaml`.
- **IA:** sí, con salida validada contra JSON Schema; repetida N veces para medir estabilidad (métrica de A).
- **Se verifica:** precisión, recall y F1 contra la lista congelada; recall primero.

CU-04 · Revisar y congelar el catálogo

- **Entrada:** `catalogo.propuesto.yaml` + FK virtuales propuestas.
- **Hace:** la persona revisa **prioritariamente las de baja confianza y las que los detectores no acuerdan**, edita, y congela. El orquestador valida el archivo (toda columna clasificada, toda estrategia existe, ninguna clave con estrategia generativa).
- **Sale:** `catalogo.vN.yaml` con hash. A partir de acá el LLM no interviene más: es lo que resuelve la no determinación.
- **Se verifica:** cantidad de columnas que la persona tuvo que cambiar (es una métrica de esfuerzo útil para la tesis).

CU-05 · Declarar restricciones de negocio

- **Entrada:** catálogo + esquema.
- **Hace:** el LLM propone restricciones candidatas (orden de fechas, rangos, sumas, unicidad compuesta, rango de partición); el responsable confirma. Las restricciones físicas del esquema se derivan solas.
- **Sale:** `restricciones.yaml` en un lenguaje declarativo propio (no SQL libre).
- **Se verifica:** las restricciones que D2 y D5 violaron (duplicados en tablas de relación, fechas fuera de partición) quedan expresadas.

CU-06 · Extraer un subconjunto

- **Entrada:** grafo, tabla(s) ancla, filtro, semilla.
- **Hace:** recorre el grafo desde el ancla, arrastra dependencias hacia arriba (padres obligatorios) y opcionalmente hacia abajo, copia completas las tablas de catálogo, resuelve ciclos.
- **Sale:** base o dump del subconjunto + reporte (% por tabla, ciclos resueltos).
- **Se verifica:** métricas de C: 0 huérfanas, % arrastrado, mismo resultado con la misma semilla. Ancla y filtro de C1/C2 ya definidos.

CU-07 · Separar el conjunto de control

- **Hace:** antes de entrenar, aparta un conjunto de entidades (p. ej. clientes) con sus dependencias, que el generador nunca ve.
- **Por qué es un caso aparte:** E3 mostró que sin control previo Anonymeter subestima el riesgo. Si se hace después, la medición no vale.
- **Sale:** identificadores del control, guardados fuera de la base de salida.

CU-08 · Transformar

- **Entrada:** subconjunto (sin control), catálogo congelado, restricciones, semilla.
- **Hace:** enrutamiento por clase de columna:

Clase	Vía
PK/FK, identificadores naturales	Determinista: tokenización con sal o regeneración con mapeo propagado
Identificadores directos con formato (nombre, email, teléfono)	Determinista con valores falsos con formato (Anonymizer o Faker), consistentes entre tablas
Catálogo	Copia tal cual
Columnas de negocio y cuasi-identificadores	Síntesis generativa (MOSTLY local)
Texto libre	CU-14
BYTEA, cifradas, contraseñas	Regla fija (L8)

Después de generar: post-proceso de claves, restricciones y remuestreo de las filas que violen alguna.

- **Sale:** base transformada + conteo de filas rechazadas/remuestreadas por restricción.
- **Se verifica:** la base carga **con todas las restricciones activas** (no como el export de Fabricate, que las desactiva).

CU-09 · Validar y aplicar el gate

- **Hace:** el evaluador (E3 es la semilla de esto) mide integridad (incluidas las FK virtuales), fidelidad (SDMetrics, SyntheRela), utilidad (suite de referencia, planes) y privacidad (Anonymeter, DCR contra control, copias exactas). Compara contra umbrales.

- **Sale:** reporte + veredicto APROBADO/RECHAZADO. **Si rechaza, la base no se entrega** (flujo alternativo FA-1).
- **IA:** ninguna. Independiente del generador.

CU-10 · Emitir el manifiesto

- Manifiesto del anexo A, con hashes de esquema, catálogo, restricciones, generador y base resultante; firmado. Se emite también si el gate rechaza (queda registro del intento).

CU-11 · Entregar la base

- Dump SQL cargable con restricciones activas en la versión de PostgreSQL fijada; solo si el gate aprobó.

CU-12 · Reproducir o auditar

- A partir de un manifiesto, vuelve a correr la validación sobre los mismos artefactos y comprueba hashes; las etapas deterministas se reejecutan y deben dar idéntico (definición de L14).

CU-13 · Deriva de esquema

- Compara el hash del esquema actual con el del último catálogo. Si cambió, lista columnas nuevas o modificadas y manda **solo esas** a CU-03; el resto del catálogo se conserva.

CU-14 · Redactar texto libre

- Detección en texto (español e inglés, con reconocedores uruguayos) y reemplazo por marcador o valor falso consistente. Requisitos de A7.

CU-15 · Pedido en lenguaje natural (futuro)

- El caso 5 de G: «prepará una copia para testing sin datos reales» → el LLM arma un **plan** (qué CU, con qué parámetros) que el orquestador valida y la persona aprueba. Interesante para mostrar la IA como orquestadora, pero no es imprescindible: dejarlo para el final si sobra tiempo.

Flujos

FP — Flujo principal (primera preparación): CU-01 → CU-02 → CU-03 → CU-04 → CU-05 → CU-06 → CU-07 → CU-08 → CU-09 → CU-10 → CU-11

FA-1 — Rechazo del gate: CU-09 rechaza → CU-10 registra → el responsable ajusta (estrategia de una columna, parámetros del generador, DP, restricciones) → vuelve a CU-08. Si la causa es una columna mal clasificada, vuelve a CU-04 y genera una versión nueva del catálogo.

FA-2 — Refresco sin cambios de esquema: CU-01 (hash igual) → reutiliza catálogo y restricciones congelados → CU-06 → … → CU-11. Ningún paso usa el LLM.

FA-3 — Refresco con deriva: CU-01 (hash distinto) → CU-13 → CU-03 solo sobre lo nuevo → CU-04 → sigue FP.

FA-4 — Degradación (informe 3.3): si el generador falla o el gate rechaza por fidelidad sin poder corregirse, la columna pasa a vía determinista (enmascarado clásico) y queda registrado en el manifiesto.

---

4. Qué casos de uso cambian según cada decisión

Si deciden...	Cambia
L1 = varios motores	CU-01 se duplica por motor; CU-08 pierde Anonymizer en los otros motores
L5 = solo Pagila	CU-03 y CU-14 se evalúan con datos contaminados y sin PII uruguaya; conviene sumar al menos la variante renombrada
L10 = el LLM ve muestras	CU-03 cambia su entrada; hay que justificarlo en el análisis de riesgos
L11 = con interfaz	CU-04 pasa a ser una pantalla; suma mucho trabajo
L12 = mini-aplicación	CU-09 suma cobertura de código
L14 = semilla estricta	CU-12 tiene que reejecutar también la síntesis y exigir igualdad

---

5. Pruebas pendientes a la luz de las limitaciones

Prueba	Sigue haciendo falta	Por qué
Congelar `columnas_sensibles_borrador.csv`	**Sí, primero**	Traba todas las de A
A2b, A6	Sí	Definen L10 (qué ve el LLM)
A3	Sí	L9 (reconocedores uruguayos)
A4 GLiNER	Sí	Es la segunda capa de la cascada de CU-03
A5 `anon.detect`	Baja	Con L1 = PostgreSQL es gratis de probar, pero no está en la cascada
B1 Anonymizer	**Sí**	Con L1 = PostgreSQL es pieza central de CU-08
B2	Sí, acotada	Valida la alternativa Faker + tokenización (L1, matiz 2)
B3 ARX	Sí, acotada	Validador de k-anonimato en CU-09
C1	**Sí**	CU-06 es componente propio; C1 es su primer prototipo

Prueba	Sigue haciendo falta	Por qué
D3 SDV	Baja	Confirma lo del informe sobre HMASynthesizer; no cambia decisiones
D4 Faker	Sí	Línea base y vía determinista de CU-08
E1, E4 SyntheRela	Sí	Completan CU-09
F1	**Sí, y sin responsable**	Define L12 y el aporte
G1	Sí	Valida el contrato JSON del LLM en CU-02/03/05
Pruebas con trial de Tonic	Según L16	
Verificar semilla de MOSTLY	**Sí, barato**	Resuelve L14
Curva de escala (2–3 tamaños)	Sí	Fija L4